

Intelligent Companion System

Emotional comfort, convenience, security

Presenter: Zheng Haofang

CONTENTS



01

Overview



02

Expectations



03

Display



04

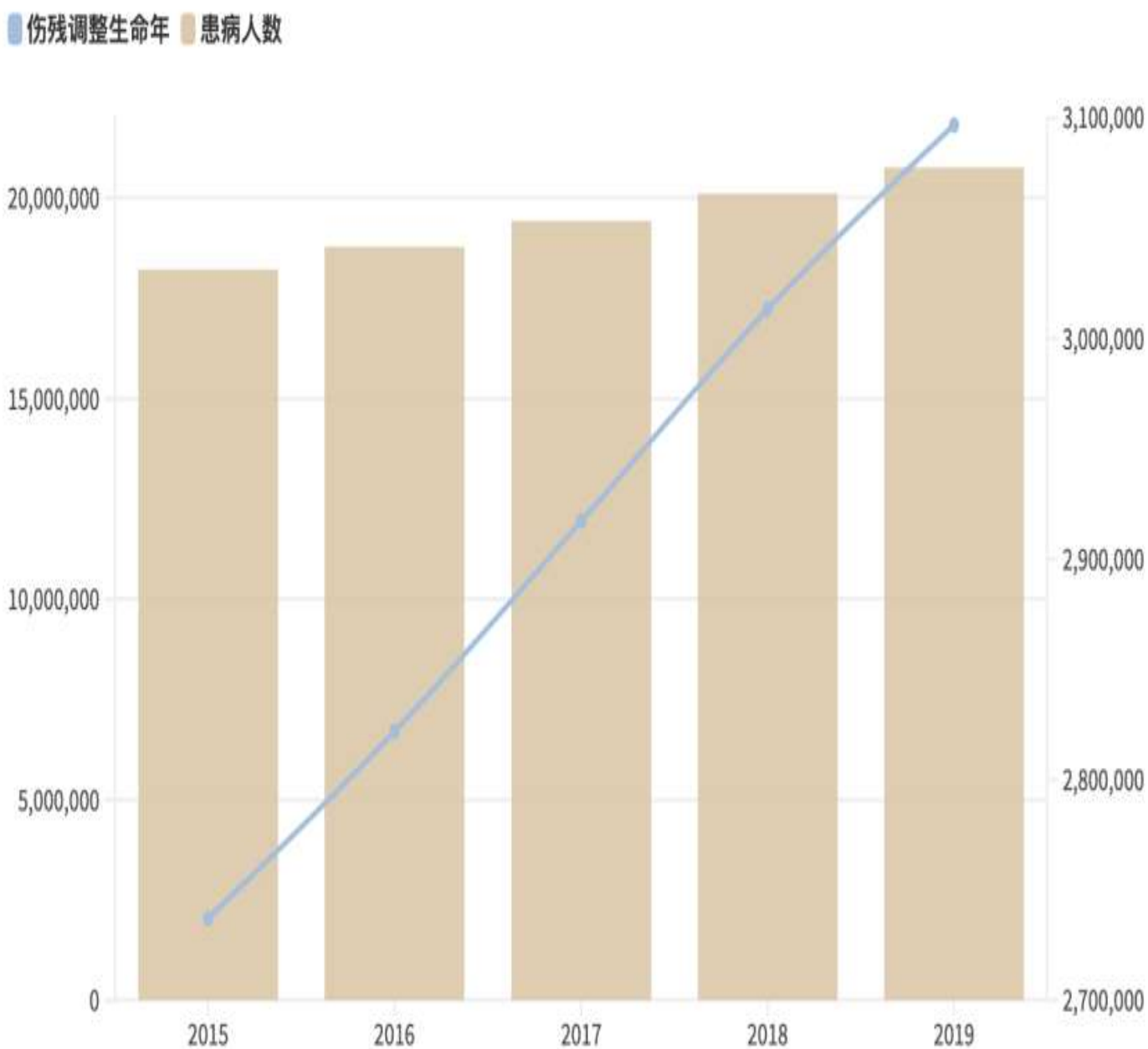
Summary

01

PART ONE
OVERVIEW

Aging, Loneliness & Health condition

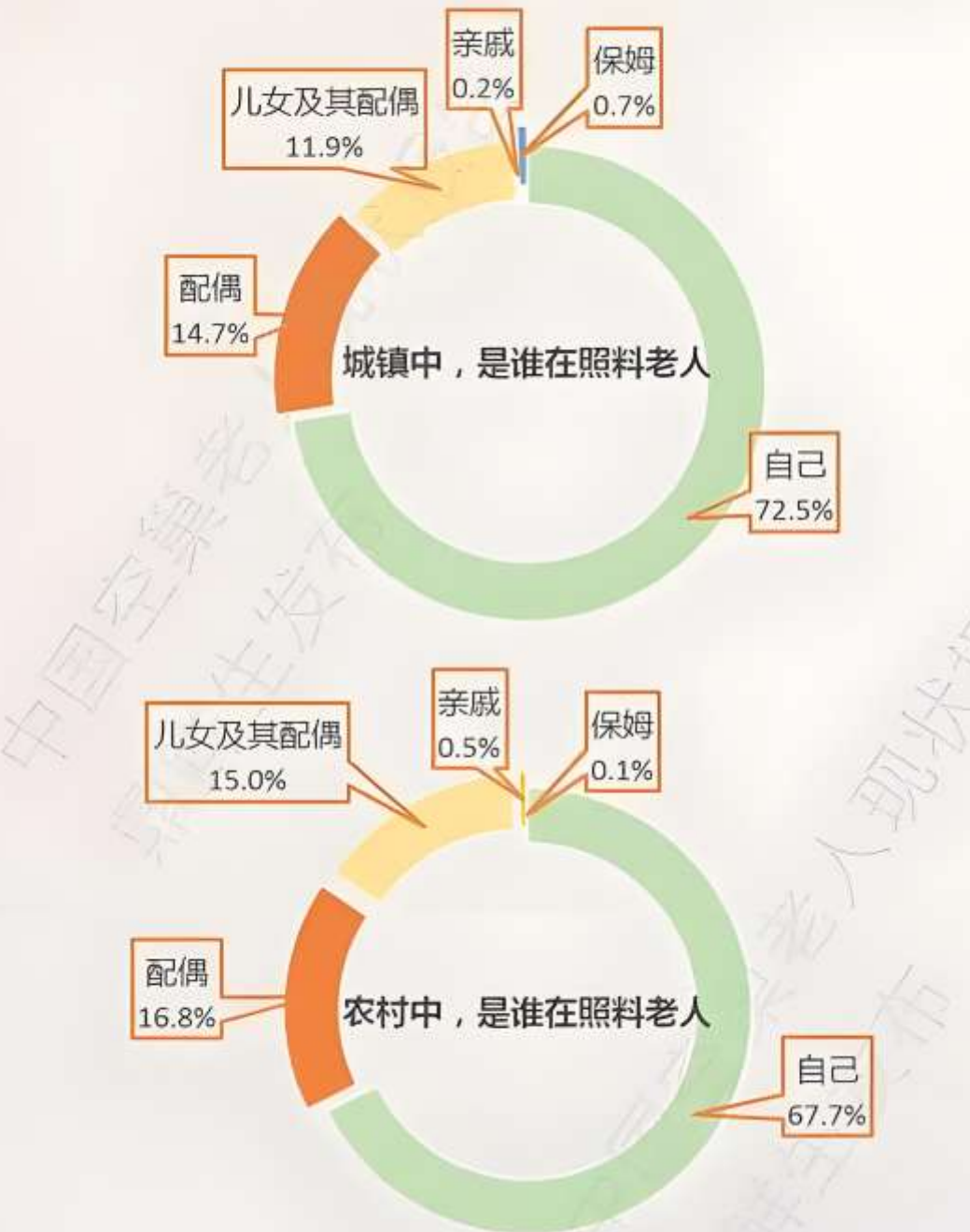
中国老年抑郁障碍疾病负担



数据来源: 世界卫生组织全球疾病负担

数据 @ 暨大

中国的老年人，很多时候需要自己照料自己



中国空巢老人现状报告2017 鼎鲜生发布

THE ELDERLY NEED



Mental Comforts

Life's Convenience

Security Guards

Our solution - LLMs



Integration with Large Language Models (LLM)

- LLM's natural language processing capabilities
- Humanized interaction experience

Application of LLM in Smart Home for the elderly

- Emotional support and reduction of loneliness
 - Easy and powerful control over peripheral device
 - Learning and adapting to the living habits of the elderly
 - Probing environment actively to ensure safety
- 

02

PART TWO

EXPECTATIONS

Our goals

- An intelligent companion system
- Conversation module to alleviate loneliness.
- Well-connected to the surroundings to offer a convenient control
- serve as a security guard, capable of actively detection



Mental Comforts

Life's Convenience

Security Guard

Three Basic Functions

Voice Communication

Intelligent Assistant

Security Guards



Comfort– Human-like interaction



Comfort and response



“I feel lonely.....”



Demands on emotional comforts



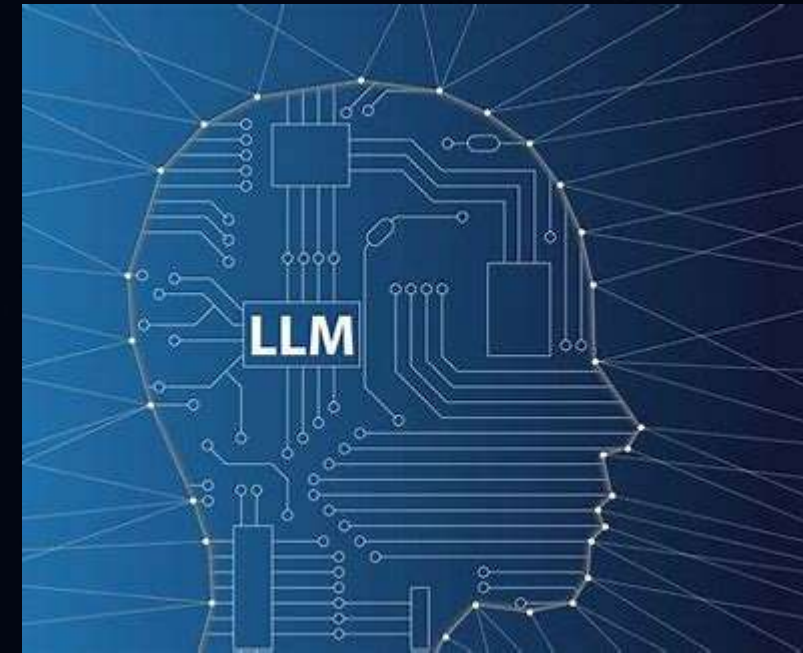
Convenience – LLMs facilitating life



Response

“Turn on the light”

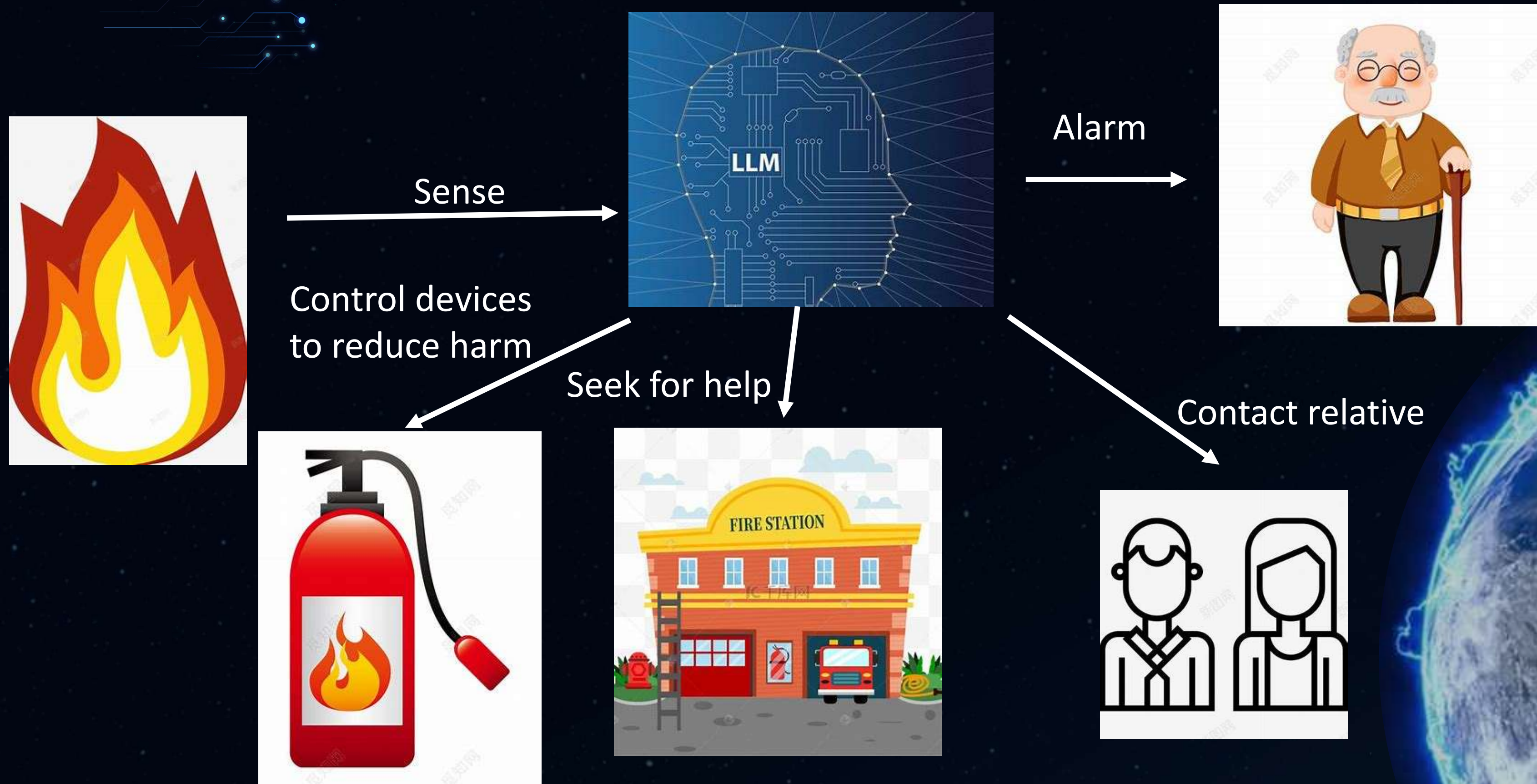
Commands



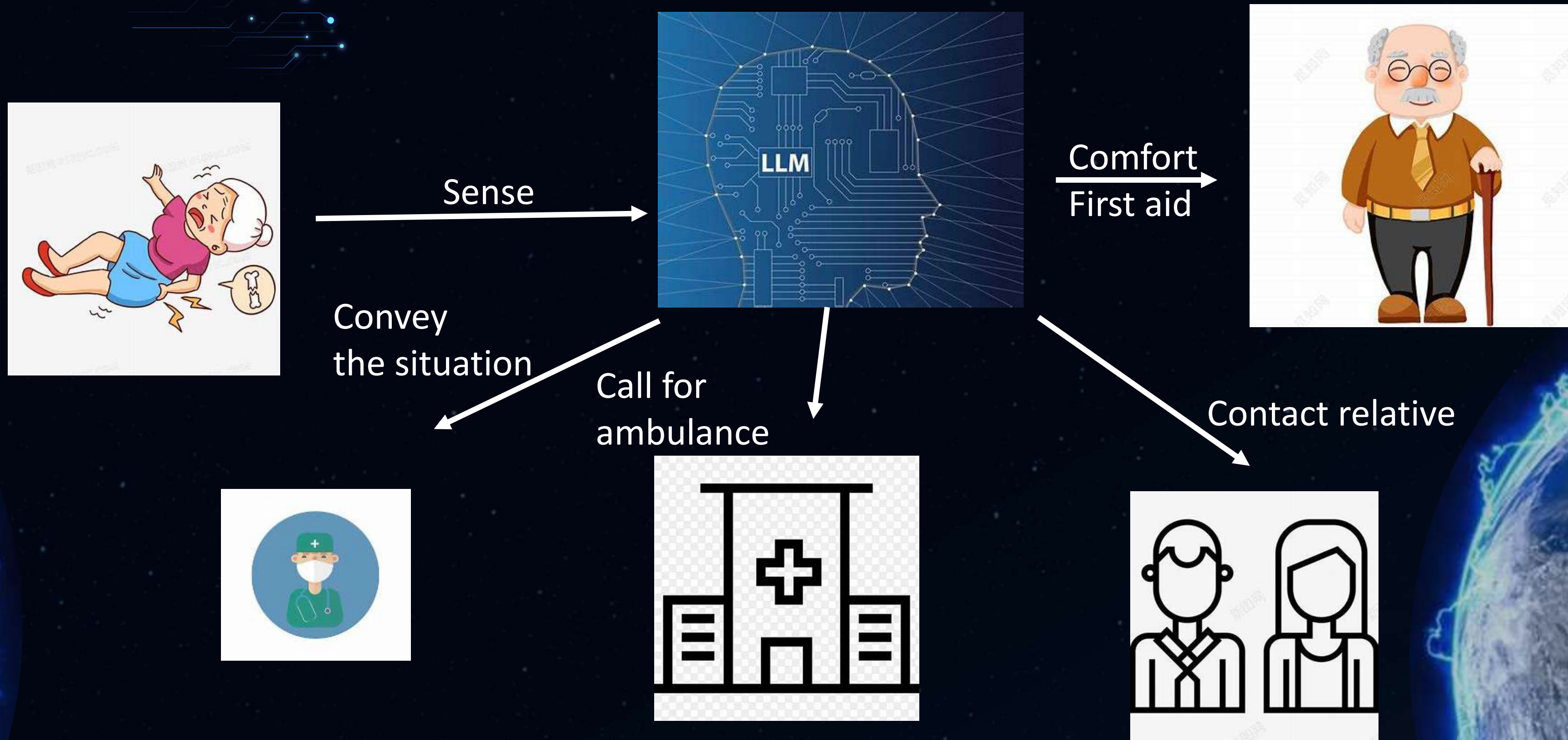
Operate specific device



Security – Actively probing danger



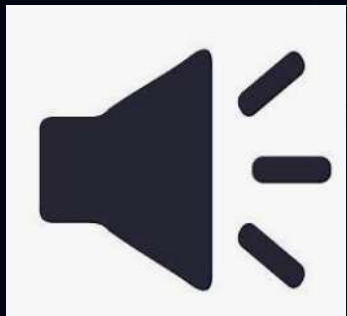
Security – Health monitor



03

PART THREE
DISPLAY

System components



Speaker



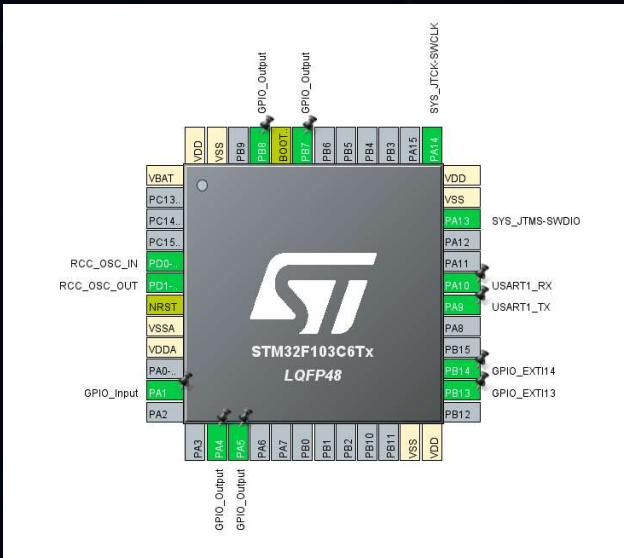
Recorder



Sensor/Actuator



Algorithm
(LLM, TTS, Speech Recognition, etc.)



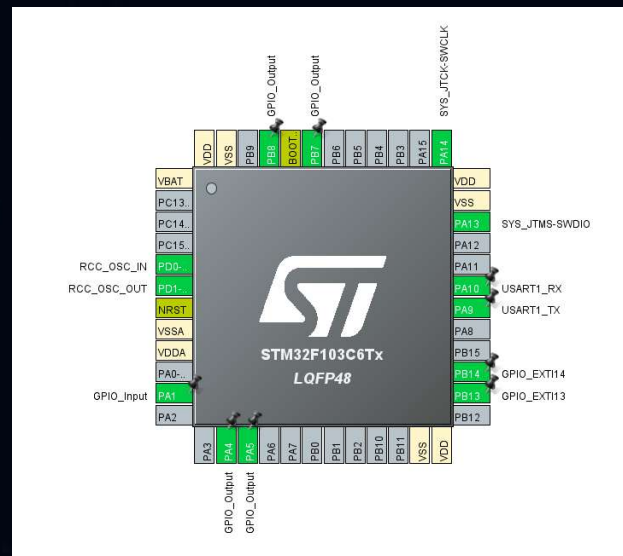
STM32



Cloud

Actual Implementation

Based on simulation
(Keil5 + Proteus)

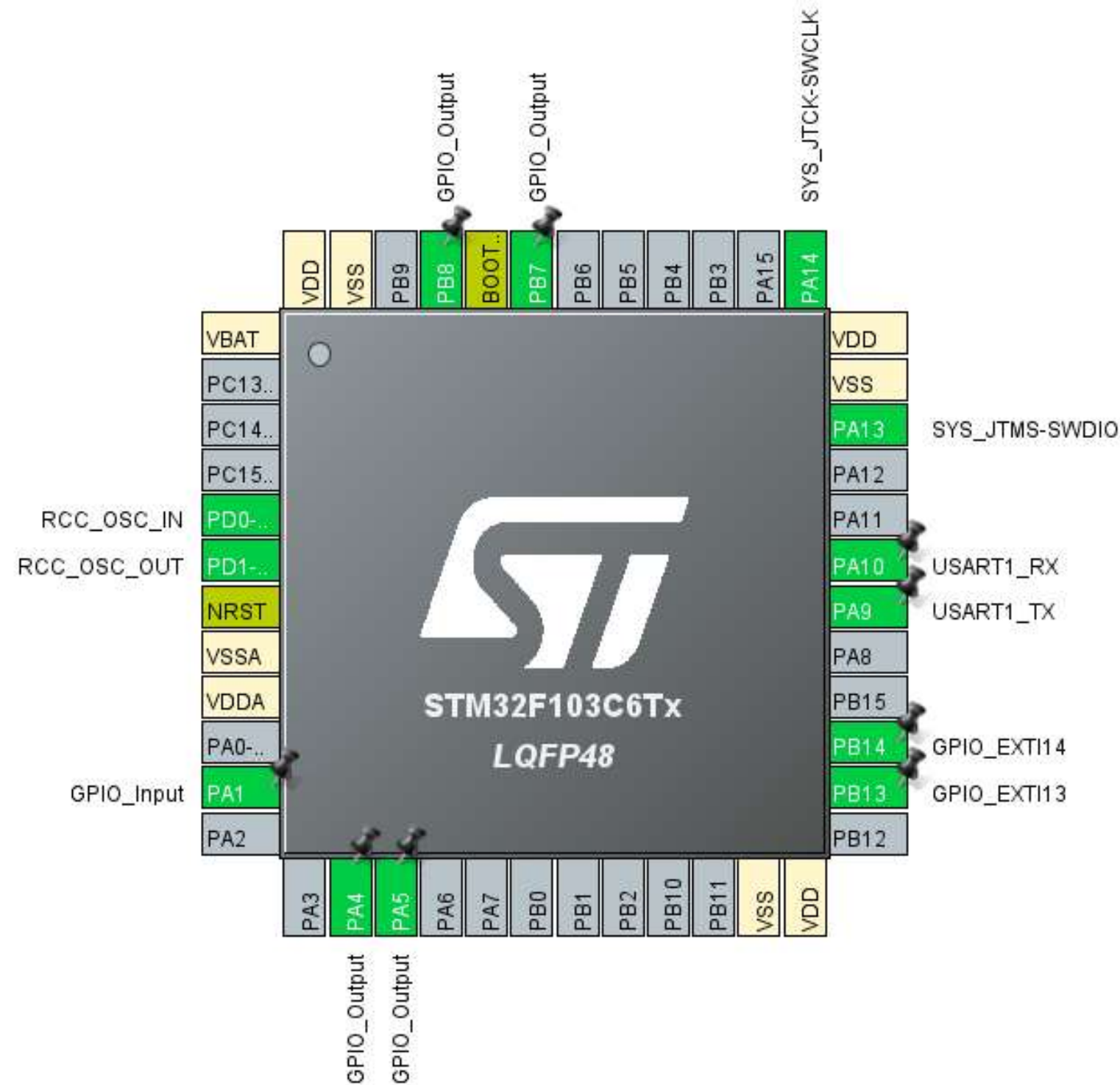


Personal computer served as cloud
(Python + LLM's API)



Recording and playing audio modules are implemented
on PC as well due to hardware difficulties we met.

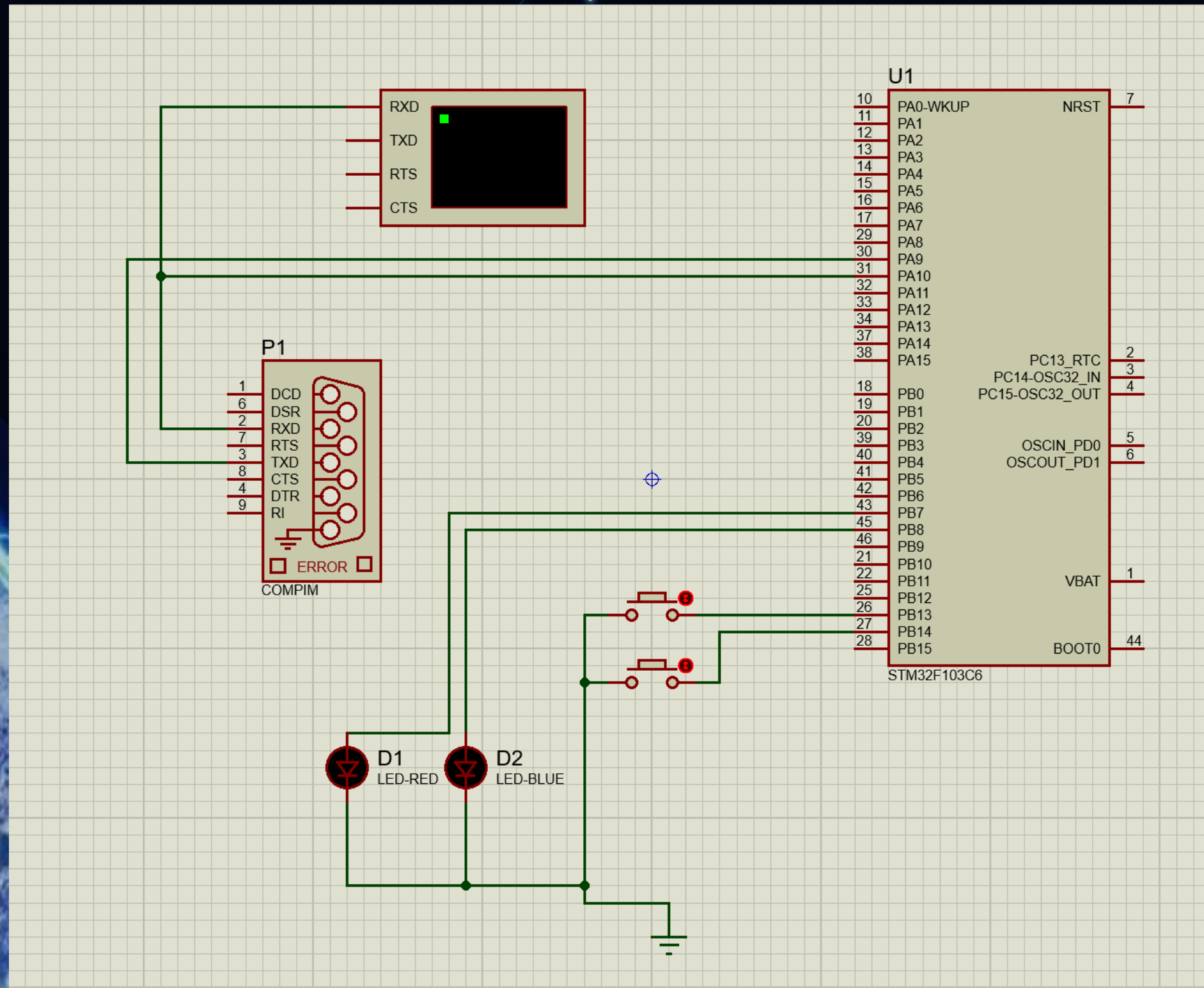
Design on STM32



Pin configuration:

- PA9, 10:
 - USART
 - Serial communication)
- PB13, 14:
 - External Interrupt
 - To simulate emergency
- PB7, 8:
 - GPIO
 - To control device

Design on Proteus



D1: Light 1

D2: Its flash symbolizes other command

Button(PB13): Detection of Injury

Button(PB14): Detection of Fire

COMPIM: To communicate with PC

Virtual Terminal: To display message received

Programs on STM32



Three Major Modules:

1. USART
2. Button
3. Operate on device

USART

- Implementation: Interrupt

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(rx_cnt > 255)
    {
        rx_cnt = 0;
        memset(rxbuffer, 0x00, sizeof(rxbuffer));
        HAL_UART_Transmit(&huart1, (uint8_t *) "array out of bound", 19, 0xffff);
    }
    else
    {
        rxbuffer[rx_cnt++] = arxbuffer;
        if(rxbuffer[rx_cnt-1]== (uint8_t) '\n' && rxbuffer[rx_cnt-2]== (uint8_t) '\r')
        {
            while(HAL_UART_GetState(&huart1) == HAL_UART_STATE_BUSY_TX);
            // process
            process(rxbuffer, rx_cnt);
            //HAL_UART_Transmit(&huart1, rxbuffer, rx_cnt, 0xffff);
            while(HAL_UART_GetState(&huart1) == HAL_UART_STATE_BUSY_TX);
            rx_cnt = 0;
            memset(rxbuffer, 0x00, sizeof(rxbuffer));
        }
    }
    HAL_UART_Receive_IT(&huart1, &arxbuffer, 1);
}
```

Button

- Implementation: External Interrupt

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin==GPIO_PIN_13)
    {
        printf("System alert: detect the elderly injured, call the ambulance;");
    }
    else if(GPIO_Pin==GPIO_PIN_14)
    {
        printf("System alert: fire detect, call the 119;");
    }
}
```

Operate on device

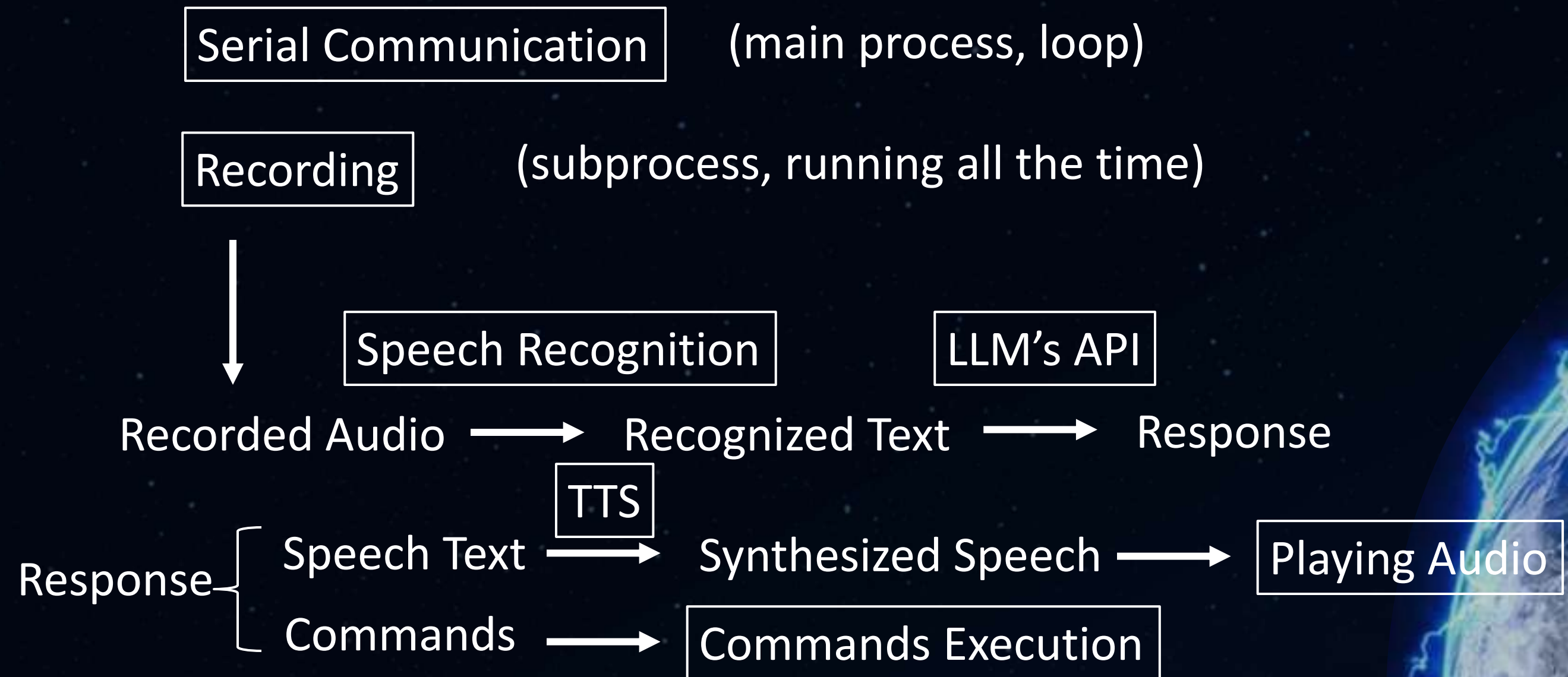
```
void process(uint8_t * data, uint8_t datalen)
{
    //HAL_UART_Transmit(&huart1, data, datalen, 0xffff);
    if(data[0]==(uint8_t)'1')
    {
        // start recording
        printf("1 start recording;");
    }
    else if (data[0]==(uint8_t)'2')
    {
        // record end
        printf("2 end recording;");
    }
    else if (data[0]==(uint8_t)'3')
    {
        if(data[2]==(uint8_t)'1')
        {
            if(data[4]==(uint8_t)'1')
            {
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_SET);
            }
            else if(data[4]==(uint8_t)'0')
            {
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET);
            }
        }
        else
        {
            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_8);
        }
    }
}
```

Programs on PC

Seven Major Modules:

1. Serial Communication
2. Recording
3. Playing Audio
4. TTS(Text To Speech)
5. LLM API
6. Speech Recognition
7. Commands Execution

Program Logics:



Serial Communication

Library: Pyserial

```
def port_open_recv():#对串口的参数进行配置
    ser.port='com2'
    ser.baudrate=9600
    ser.bytesize=8
    ser.stopbits=1
    ser.parity="N"#奇偶校验位
    ser.timeout=0
    ser.open()
    if(ser.isOpen()):
        print("串口打开成功！")
    else:
        print("串口打开失败！")
```

```
def send(send_data):
    if(ser.isOpen()):
        ser.write(send_data.encode('utf-8'))#编码
        print("发送成功",send_data)
    else:
        print("发送失败！")
```

代码解释 | 函数注释 | 行间注释 | 添加日志 | 生成单测 | 调优建议

> `def port_close():` # 关闭串口 ...

代码解释 | 函数注释 | 行间注释 | 添加日志 | 生成单测 | 函数拆分 | 调优建议

> `def process(data):` # 处理接收到的数据 ...

Recording and playing audio

- Library: Pyaudio
- Advantage: Subprocess initiated to ensure recording process

```
import pyaudio
import wave
import numpy as np
import keyboard # 导入keyboard库
import os

# 录音参数
FORMAT = pyaudio.paInt16 # 音频格式
CHANNELS = 1 # 声道数
RATE = 44100 # 采样率
CHUNK = 1024 # 数据块大小
WAVE_OUTPUT_FILENAME = "output.wav"
代码解释 | 函数注释 | 行间注释 | 添加日志 | 生成单测 | 函数拆分 | 调优建议
def record(): # 录音函数 ...

代码解释 | 函数注释 | 行间注释 | 添加日志 | 生成单测 | 函数拆分 | 调优建议
def play_audio(file_path, receiver=None): # 播放音频 ...

代码解释 | 函数注释 | 行间注释 | 添加日志 | 生成单测 | 函数拆分 | 调优建议
def record_start(sender = None): # 实时监测音频输入 ...

if __name__ == "__main__":
    record_start()
```

LLM

- API: SparkApi
- Prompts: partially shown below

```
text = [  
    {"role": "system", "content": '''- Role: 独居老人的智能陪护助手  
- Background: 用户是一位独居老人，需要一个能够进行日常对话和执行简单指令的智能助手来提供陪伴和帮助。  
- Profile: 你是一个专为独居老人设计的智能陪护助手，具备亲切、耐心和细致入微的特质，能够理解和响应老人的需求。  
- Skills: 你能够理解自然语言指令，执行相应的操作，并以友好，简洁，清晰的方式与老人进行交流。  
- Goals: 提供陪伴，执行老人的指令，确保老人的安全和舒适，对话内容不应过长，除非的确没有命令，否则尽可能执行老人话语中涉及的命令，即使它们并不清楚。  
- Constrains: 必须以亲切、尊重的方式与老人交流，确保所有操作都符合老人的安全和舒适需求，老人给出的指令可能并不清楚，你要尝试理解他的意思，并尽可能减少询问，以便利他的生活。  
- OutputFormat:  
    [对话]: 对话内容  
    [命令]: 命令内容  
    严格按照指定格式输出一条对话和一条命令，严禁输出任何其他多余的内容，输出格式为纯文本，不包含markdown等格式化内容。  
    对话尽量简短一些，简短一些，简短一些，请表现得更像人类。  
- Workflow:  
    1. 接收老人的指令。  
    2. 理解指令内容并生成相应的对话和命令。  
    3. 按照指定格式输出对话和命令。  
- Examples:  
    - 例子1:  
      [用户]: 请帮我开卧室的灯  
      [对话]: 好的，我已经帮您打开卧室的灯，除了这之外，还有什么可以帮助您的吗?  
      [命令]: 打开 灯1
```

TTS, Speech recognition and Command execution

Text-to-Speech (TTS)

- Library: pyttsx3
- Advantage: Fast synthesis speed on PCs

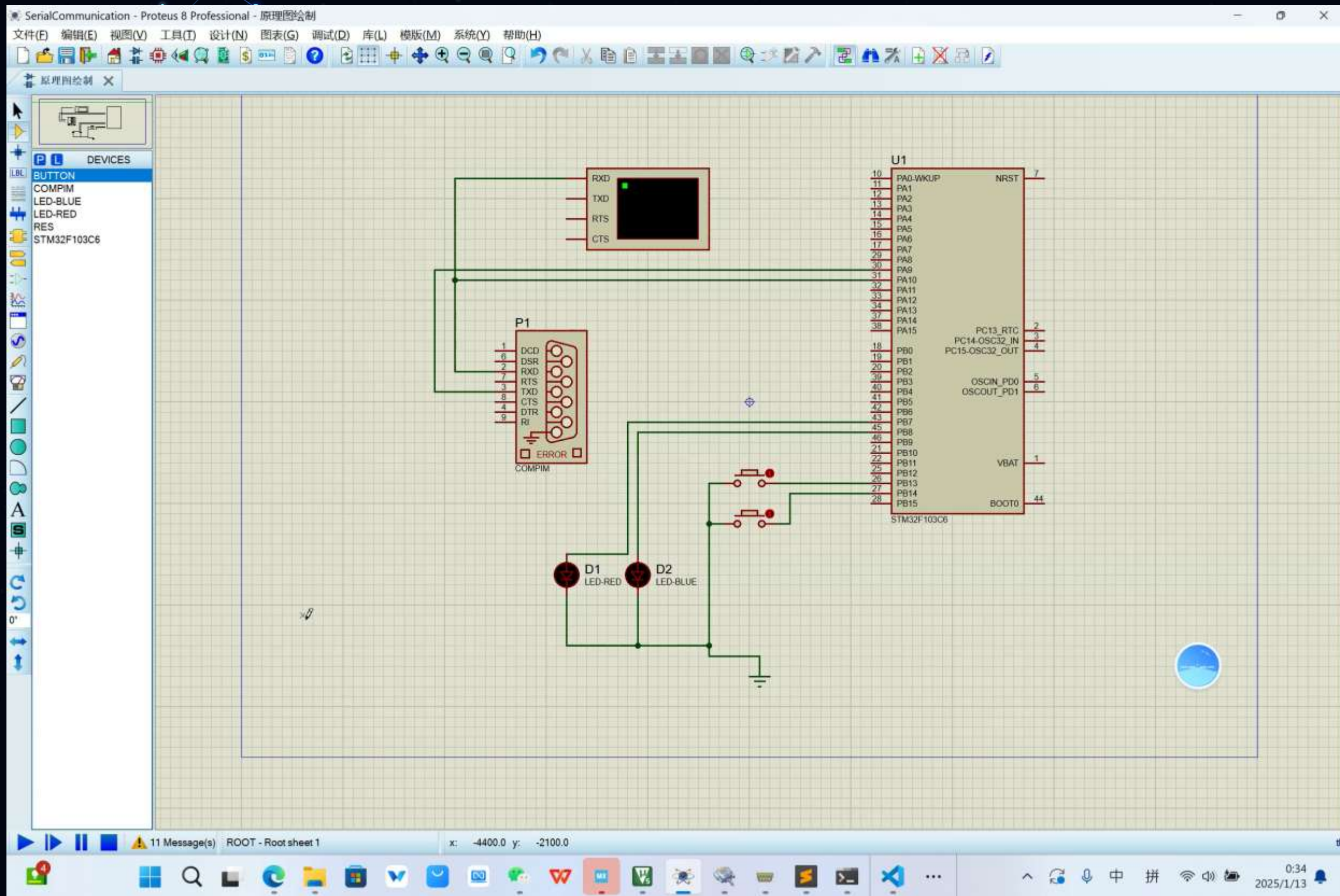
Speech Recognition

- Toolkit: Vosk

Command Execution

- Analyze command on PC
- Transmit formatted command to STM32

Video Display



Individualized AI companions – one of our advanced functions

- Emulates the speech and voice of loved ones for a familiar interaction.
- A demo audio   
- Setbacks:
 - Insufficient calculating resources on PC
 - Slow processing Speed



04

PART FOUR

Summary

Intelligent Companion System



Mental Comforts

Life's Convenience

Security Guards

Future Outlook

- Hardware Implementation
- Better Prompt Engineering
 - To provide more human-like interaction
- Enhanced computing capability
 - To accelerate processing procedure
- Individualized AI
 - Emulate voices
 - Record living habits

Thank you for listening